JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class BitSet

java.lang.Object
    java.util.BitSet

All Implemented Interfaces:

Serializable, Cloneable

```
public class BitSet
extends Object
implements Cloneable, Serializable
```

This class implements a vector of bits that grows as needed. Each component of the bit set has a `boolean` value. The bits of a `BitSet` are indexed by nonnegative integers. Individual indexed bits can be examined, set, or cleared. One `BitSet` may be used to modify the contents of another `BitSet` through logical AND, logical inclusive OR, and logical exclusive OR operations.

By default, all bits in the set initially have the value `false`.

Every bit set has a current size, which is the number of bits of space currently in use by the bit set. Note that the size is related to the implementation of a bit set, so it may change with implementation. The length of a bit set relates to logical length of a bit set and is defined independently of implementation.

Unless otherwise noted, passing a null parameter to any of the methods in a `BitSet` will result in a `NullPointerException`.

A `BitSet` is not safe for multithreaded use without external synchronization.

Since:

1.0

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`BitSet()`	Creates a new bit set.
`BitSet(int nbits)`	Creates a bit set whose initial size is large enough to explicitly represent bits with indices in the range `0` through `nbits-1`.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
void	**and**(BitSet set)	Performs a logical **AND** of this target bit set with the argument bit set.
void	**andNot**(BitSet set)	Clears all of the bits in this BitSet whose corresponding bit is set in the specified BitSet.
int	**cardinality**()	Returns the number of bits set to true in this BitSet.
void	**clear**()	Sets all of the bits in this BitSet to false.
void	**clear**(int bitIndex)	Sets the bit specified by the index to false.
void	**clear**(int fromIndex, int toIndex)	Sets the bits from the specified fromIndex (inclusive) to the specified toIndex (exclusive) to false.
Object	**clone**()	Cloning this BitSet produces a new BitSet that is equal to it.
boolean	**equals**(Object obj)	Compares this bit set against the specified object.
void	**flip**(int bitIndex)	Sets the bit at the specified index to the complement of its current value.
void	**flip**(int fromIndex, int toIndex)	Sets each bit from the specified fromIndex (inclusive) to the specified toIndex

		(exclusive) to the complement of its current value.
boolean	**get**(int bitIndex)	Returns the value of the bit with the specified index.
BitSet	**get**(int fromIndex, int toIndex)	Returns a new BitSet composed of bits from this BitSet from fromIndex (inclusive) to toIndex (exclusive).
int	**hashCode**()	Returns the hash code value for this bit set.
boolean	**intersects**(BitSet set)	Returns true if the specified BitSet has any bits set to true that are also set to true in this BitSet.
boolean	**isEmpty**()	Returns true if this BitSet contains no bits that are set to true.
int	**length**()	Returns the "logical size" of this BitSet: the index of the highest set bit in the BitSet plus one.
int	**nextClearBit**(int fromIndex)	Returns the index of the first bit that is set to false that occurs on or after the specified starting index.
int	**nextSetBit**(int fromIndex)	Returns the index of the first bit that is set to true that occurs on or

		after the specified starting index.
void	or(BitSet set)	Performs a logical **OR** of this bit set with the bit set argument.
int	previousClearBit(int fromIndex)	Returns the index of the nearest bit that is set to `false` that occurs on or before the specified starting index.
int	previousSetBit(int fromIndex)	Returns the index of the nearest bit that is set to `true` that occurs on or before the specified starting index.
void	set(int bitIndex)	Sets the bit at the specified index to `true`.
void	set(int bitIndex, boolean value)	Sets the bit at the specified index to the specified value.
void	set(int fromIndex, int toIndex)	Sets the bits from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to `true`.
void	set(int fromIndex, int toIndex, boolean value)	Sets the bits from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to the specified value.
int	size()	Returns the number of bits of space actually in use by this `BitSet` to represent bit values.

IntStream	**stream**()	Returns a stream of indices for which this BitSet contains a bit in the set state.
byte[]	**toByteArray**()	Returns a new byte array containing all the bits in this bit set.
long[]	**toLongArray**()	Returns a new long array containing all the bits in this bit set.
String	**toString**()	Returns a string representation of this bit set.
static BitSet	**valueOf**(byte[] bytes)	Returns a new bit set containing all the bits in the given byte array.
static BitSet	**valueOf**(long[] longs)	Returns a new bit set containing all the bits in the given long array.
static BitSet	**valueOf**(ByteBuffer bb)	Returns a new bit set containing all the bits in the given byte buffer between its position and limit.
static BitSet	**valueOf**(LongBuffer lb)	Returns a new bit set containing all the bits in the given long buffer between its position and limit.
void	**xor**(BitSet set)	Performs a logical **XOR** of this bit set with the bit set argument.

Methods declared in class Object

`finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

BitSet

```
public BitSet()
```

Creates a new bit set. All bits are initially `false`.

BitSet

```
public BitSet(int nbits)
```

Creates a bit set whose initial size is large enough to explicitly represent bits with indices in the range `0` through `nbits-1`. All bits are initially `false`.

Parameters:

`nbits` - the initial size of the bit set

Throws:

`NegativeArraySizeException` - if the specified initial size is negative

Method Details

valueOf

```
public static BitSet valueOf(long[] longs)
```

Returns a new bit set containing all the bits in the given long array.

More precisely,
`BitSet.valueOf(longs).get(n) == ((longs[n/64] & (1L<<(n%64))) != 0)`
for all `n < 64 * longs.length`.

This method is equivalent to `BitSet.valueOf(LongBuffer.wrap(longs))`.

Parameters:

`longs` - a long array containing a little-endian representation of a sequence of bits to be used as the initial bits of the new bit set

Returns:

a `BitSet` containing all the bits in the long array

Since:

1.7

valueOf

```
public static BitSet valueOf(LongBuffer lb)
```

Returns a new bit set containing all the bits in the given long buffer between its position and limit.

More precisely,
`BitSet.valueOf(lb).get(n) == ((lb.get(lb.position()+n/64) & (1L<<(n%64))) != 0)`
for all `n < 64 * lb.remaining()`.

The long buffer is not modified by this method, and no reference to the buffer is retained by the bit set.

Parameters:

`lb` - a long buffer containing a little-endian representation of a sequence of bits between its position and limit, to be used as the initial bits of the new bit set

Returns:

a `BitSet` containing all the bits in the buffer in the specified range

Since:

1.7

valueOf

```
public static BitSet valueOf(byte[] bytes)
```

Returns a new bit set containing all the bits in the given byte array.

More precisely,
`BitSet.valueOf(bytes).get(n) == ((bytes[n/8] & (1<<(n%8))) != 0)`
for all `n < 8 * bytes.length`.

This method is equivalent to `BitSet.valueOf(ByteBuffer.wrap(bytes))`.

Parameters:

`bytes` - a byte array containing a little-endian representation of a sequence of bits to be used as the initial bits of the new bit set

Returns:

a `BitSet` containing all the bits in the byte array

Since:

1.7

valueOf

```
public static BitSet valueOf(ByteBuffer bb)
```

Returns a new bit set containing all the bits in the given byte buffer between its position and limit.

More precisely,
`BitSet.valueOf(bb).get(n) == ((bb.get(bb.position()+n/8) & (1<<(n%8))) != 0)`
for all `n < 8 * bb.remaining()`.

The byte buffer is not modified by this method, and no reference to the buffer is retained by the bit set.

Parameters:

`bb` - a byte buffer containing a little-endian representation of a sequence of bits between its position and limit, to be used as the initial bits of the new bit set

Returns:

a `BitSet` containing all the bits in the buffer in the specified range

Since:

1.7

toByteArray

```
public byte[] toByteArray()
```

Returns a new byte array containing all the bits in this bit set.

More precisely, if
`byte[] bytes = s.toByteArray();`
then `bytes.length == (s.length()+7)/8` and
`s.get(n) == ((bytes[n/8] & (1<<(n%8))) != 0)`
for all `n < 8 * bytes.length`.

Returns:

a byte array containing a little-endian representation of all the bits in this bit set

Since:

1.7

toLongArray

```
public long[] toLongArray()
```

Returns a new long array containing all the bits in this bit set.

More precisely, if
`long[] longs = s.toLongArray();`
then `longs.length == (s.length()+63)/64` and
`s.get(n) == ((longs[n/64] & (1L<<(n%64))) != 0)`
for all `n < 64 * longs.length`.

Returns:

a long array containing a little-endian representation of all the bits in this bit set

Since:

1.7

flip

```
public void flip(int bitIndex)
```

Sets the bit at the specified index to the complement of its current value.

Parameters:

`bitIndex` - the index of the bit to flip

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.4

flip

```
public void flip(int fromIndex,
                 int toIndex)
```

Sets each bit from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to the complement of its current value.

Parameters:

`fromIndex` - index of the first bit to flip

`toIndex` - index after the last bit to flip

Throws:

`IndexOutOfBoundsException` - if `fromIndex` is negative, or `toIndex` is negative, or `fromIndex` is larger than `toIndex`

Since:

1.4

set

```
public void set(int bitIndex)
```

Sets the bit at the specified index to `true`.

Parameters:

`bitIndex` - a bit index

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.0

set

```
public void set(int bitIndex,
                boolean value)
```

Sets the bit at the specified index to the specified value.

Parameters:

`bitIndex` - a bit index

`value` - a boolean value to set

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.4

set

```
public void set(int fromIndex,
                int toIndex)
```

Sets the bits from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to `true`.

Parameters:

`fromIndex` - index of the first bit to be set

`toIndex` - index after the last bit to be set

Throws:

`IndexOutOfBoundsException` - if `fromIndex` is negative, or `toIndex` is negative, or `fromIndex` is larger than `toIndex`

Since:

1.4

set

```
public void set(int fromIndex,
                int toIndex,
                boolean value)
```

Sets the bits from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to the specified value.

Parameters:

`fromIndex` - index of the first bit to be set

`toIndex` - index after the last bit to be set

`value` - value to set the selected bits to

Throws:

`IndexOutOfBoundsException` - if `fromIndex` is negative, or `toIndex` is negative, or `fromIndex` is larger than `toIndex`

Since:

1.4

clear

```
public void clear(int bitIndex)
```

Sets the bit specified by the index to `false`.

Parameters:

`bitIndex` - the index of the bit to be cleared

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.0

clear

```
public void clear(int fromIndex,
                  int toIndex)
```

Sets the bits from the specified `fromIndex` (inclusive) to the specified `toIndex` (exclusive) to `false`.

Parameters:

`fromIndex` - index of the first bit to be cleared

`toIndex` - index after the last bit to be cleared

Throws:

IndexOutOfBoundsException - if fromIndex is negative, or toIndex is negative, or fromIndex is larger than toIndex

Since:

1.4

clear

```
public void clear()
```

Sets all of the bits in this BitSet to false.

Since:

1.4

get

```
public boolean get(int bitIndex)
```

Returns the value of the bit with the specified index. The value is true if the bit with the index bitIndex is currently set in this BitSet; otherwise, the result is false.

Parameters:

bitIndex - the bit index

Returns:

the value of the bit with the specified index

Throws:

IndexOutOfBoundsException - if the specified index is negative

get

```
public BitSet get(int fromIndex,
                  int toIndex)
```

Returns a new BitSet composed of bits from this BitSet from fromIndex (inclusive) to toIndex (exclusive).

Parameters:

fromIndex - index of the first bit to include

toIndex - index after the last bit to include

Returns:

a new BitSet from a range of this BitSet

Throws:

IndexOutOfBoundsException - if fromIndex is negative, or toIndex is negative, or fromIndex is larger than toIndex

Since:

1.4

nextSetBit

`public int nextSetBit(int fromIndex)`

Returns the index of the first bit that is set to `true` that occurs on or after the specified starting index. If no such bit exists then `-1` is returned.

To iterate over the `true` bits in a `BitSet`, use the following loop:

```
for (int i = bs.nextSetBit(0); i >= 0; i = bs.nextSetBit(i+1)) {
    // operate on index i here
    if (i == Integer.MAX_VALUE) {
        break; // or (i+1) would overflow
    }
}
```

Parameters:

`fromIndex` - the index to start checking from (inclusive)

Returns:

the index of the next set bit, or `-1` if there is no such bit

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.4

nextClearBit

`public int nextClearBit(int fromIndex)`

Returns the index of the first bit that is set to `false` that occurs on or after the specified starting index.

Parameters:

`fromIndex` - the index to start checking from (inclusive)

Returns:

the index of the next clear bit

Throws:

`IndexOutOfBoundsException` - if the specified index is negative

Since:

1.4

previousSetBit

`public int previousSetBit(int fromIndex)`

Returns the index of the nearest bit that is set to `true` that occurs on or before the specified starting index. If no such bit exists, or if `-1` is given as the starting index, then `-1` is returned.

To iterate over the `true` bits in a `BitSet`, use the following loop:

```
for (int i = bs.length(); (i = bs.previousSetBit(i-1)) >= 0; ) {
    // operate on index i here
}
```

Parameters:

`fromIndex` - the index to start checking from (inclusive)

Returns:

the index of the previous set bit, or `-1` if there is no such bit

Throws:

`IndexOutOfBoundsException` - if the specified index is less than `-1`

Since:

1.7

previousClearBit

`public int previousClearBit(int fromIndex)`

Returns the index of the nearest bit that is set to `false` that occurs on or before the specified starting index. If no such bit exists, or if `-1` is given as the starting index, then `-1` is returned.

Parameters:

`fromIndex` - the index to start checking from (inclusive)

Returns:

the index of the previous clear bit, or `-1` if there is no such bit

Throws:

`IndexOutOfBoundsException` - if the specified index is less than `-1`

Since:

1.7

length

`public int length()`

Returns the "logical size" of this `BitSet`: the index of the highest set bit in the `BitSet` plus one. Returns zero if the `BitSet` contains no set bits.

Returns:

the logical size of this `BitSet`

Since:

1.2

isEmpty

```
public boolean isEmpty()
```

Returns true if this `BitSet` contains no bits that are set to `true`.

Returns:

boolean indicating whether this `BitSet` is empty

Since:

1.4

intersects

```
public boolean intersects(BitSet set)
```

Returns true if the specified `BitSet` has any bits set to `true` that are also set to `true` in this `BitSet`.

Parameters:

`set` - `BitSet` to intersect with

Returns:

boolean indicating whether this `BitSet` intersects the specified `BitSet`

Since:

1.4

cardinality

```
public int cardinality()
```

Returns the number of bits set to `true` in this `BitSet`.

Returns:

the number of bits set to `true` in this `BitSet`

Since:

1.4

and

`public void and(BitSet set)`

Performs a logical **AND** of this target bit set with the argument bit set. This bit set is modified so that each bit in it has the value `true` if and only if it both initially had the value `true` and the corresponding bit in the bit set argument also had the value `true`.

Parameters:

`set` - a bit set

or

`public void or(BitSet set)`

Performs a logical **OR** of this bit set with the bit set argument. This bit set is modified so that a bit in it has the value `true` if and only if it either already had the value `true` or the corresponding bit in the bit set argument has the value `true`.

Parameters:

`set` - a bit set

xor

`public void xor(BitSet set)`

Performs a logical **XOR** of this bit set with the bit set argument. This bit set is modified so that a bit in it has the value `true` if and only if one of the following statements holds:

- The bit initially has the value `true`, and the corresponding bit in the argument has the value `false`.
- The bit initially has the value `false`, and the corresponding bit in the argument has the value `true`.

Parameters:

`set` - a bit set

andNot

`public void andNot(BitSet set)`

Clears all of the bits in this `BitSet` whose corresponding bit is set in the specified `BitSet`.

Parameters:

`set` - the `BitSet` with which to mask this `BitSet`

Since:

1.2

hashCode

```
public int hashCode()
```

Returns the hash code value for this bit set. The hash code depends only on which bits are set within this `BitSet`.

The hash code is defined to be the result of the following calculation:

```
public int hashCode() {
    long h = 1234;
    long[] words = toLongArray();
    for (int i = words.length; --i >= 0; )
        h ^= words[i] * (i + 1);
    return (int)((h >> 32) ^ h);
}
```

Note that the hash code changes if the set of bits is altered.

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this bit set

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

size

```
public int size()
```

Returns the number of bits of space actually in use by this `BitSet` to represent bit values. The maximum element in the set is the size - 1st element.

Returns:

the number of bits currently in this bit set

equals

```
public boolean equals(Object obj)
```

Compares this bit set against the specified object. The result is `true` if and only if the argument is not `null` and is a `BitSet` object that has exactly the same set of bits set to `true` as this bit set. That is, for every nonnegative `int` index `k`,

```
((BitSet)obj).get(k) == this.get(k)
```

must be true. The current sizes of the two bit sets are not compared.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to compare with

Returns:

`true` if the objects are the same; `false` otherwise

See Also:

`size()`

clone

`public Object clone()`

Cloning this `BitSet` produces a new `BitSet` that is equal to it. The clone of the bit set is another bit set that has exactly the same bits set to `true` as this bit set.

Overrides:

`clone` in class `Object`

Returns:

a clone of this bit set

See Also:

`size()`

toString

`public String toString()`

Returns a string representation of this bit set. For every index for which this `BitSet` contains a bit in the set state, the decimal representation of that index is included in the result. Such indices are listed in order from lowest to highest, separated by ", " (a comma and a space) and surrounded by braces, resulting in the usual mathematical notation for a set of integers.

Example:

```
BitSet drPepper = new BitSet();
```

Now `drPepper.toString()` returns `"{}"`.

```
drPepper.set(2);
```

Now `drPepper.toString()` returns `"{2}"`.

```
drPepper.set(4);
drPepper.set(10);
```

Now `drPepper.toString()` returns `"{2, 4, 10}"`.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this bit set

stream

`public IntStream stream()`

Returns a stream of indices for which this `BitSet` contains a bit in the set state. The indices are returned in order, from lowest to highest. The size of the stream is the number of bits in the set state, equal to the value returned by the `cardinality()` method.

The stream binds to this bit set when the terminal stream operation commences (specifically, the spliterator for the stream is *late-binding*). If the bit set is modified during that operation then the result is undefined.

Returns:

a stream of integers representing set indices

Since:

1.8

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.